

Ring Bus ESL 模型方案与架构评审稿

1. 建模目标

本模型用于在 ESL 环境中描述多发起端、多目标端 Ring Bus 的功能、时延、带宽、流控和拥塞行为。模型不依赖 SoC 顶层，可以独立挂接事务发起端和存储/外设目标端，也可以作为互连组件集成到更大的系统模型中。

核心目标如下：

- 支持参数化数量的 master 和 target；
- 支持读、写命令、写数据和响应的完整事务闭环；
- 支持地址译码、LINEAR 和 XOR_HASH 交织；
- 建模逐跳传播延迟、链路序列化带宽、FIFO、接口延迟和反压；
- 支持 master、全局、target 多级 OSD/credit 限制；
- 支持 target 访问延迟、吞吐限制、带宽 token 和热点惩罚；
- 输出吞吐、延迟、公平性、资源占用和瓶颈统计；
- 保证配置有效性、事务守恒、无数据丢失和可检测的系统排空状态。

本模型定位为 transaction-level、cycle-approximate ESL 性能模型，不替代 RTL 的 flit/VC 级 cycle-accurate 仿真。

2. 设计原则

2.1 与 SoC 解耦

Ring Bus 只依赖通用事务接口和时钟，不依赖 CPU、Cache、DMA 或特定 SoC 顶层。外部模块通过 master/target attach 接口连接，因此同一个 Ring Bus 可以在独立测试环境或系统模型中复用。

2.2 功能与性能状态统一

请求、响应、OSD、FIFO、链路占用和 target credit 使用同一套事务状态推进。性能延迟不是在功能模型完成后额外累加，而是直接决定事务何时能够向下一级移动。

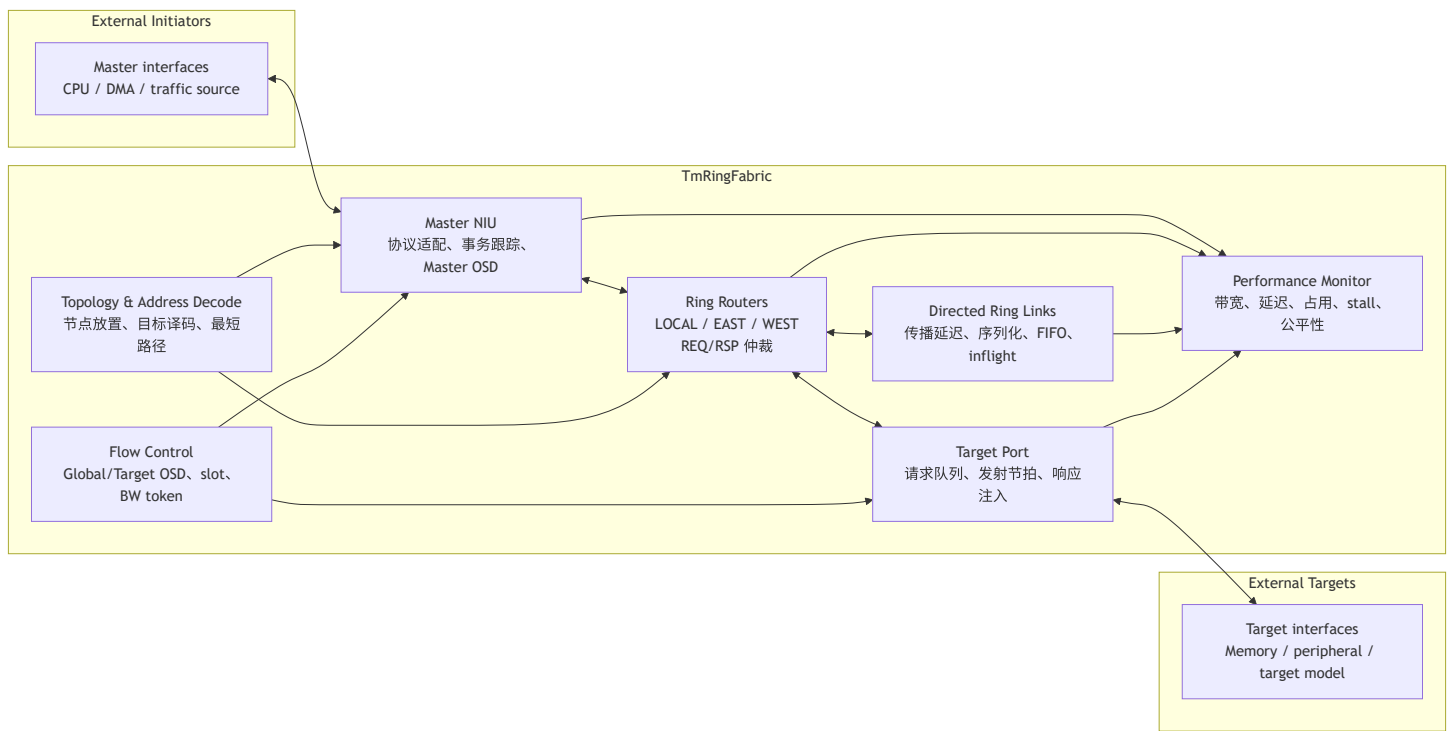
2.3 参数显式化

拓扑、交织、延迟、带宽、FIFO、OSD 和 target 能力均通过配置描述。模型代码不应包含固定 endpoint 数量、固定地址映射或固定性能参数。

2.4 反压无丢包

下一级不可接收时，上一级保持 payload valid，不弹出当前事务。事务只在下一级成功接收后提交，从而保证反压场景下的数据完整性。

3. 总体架构



3.1 模块职责

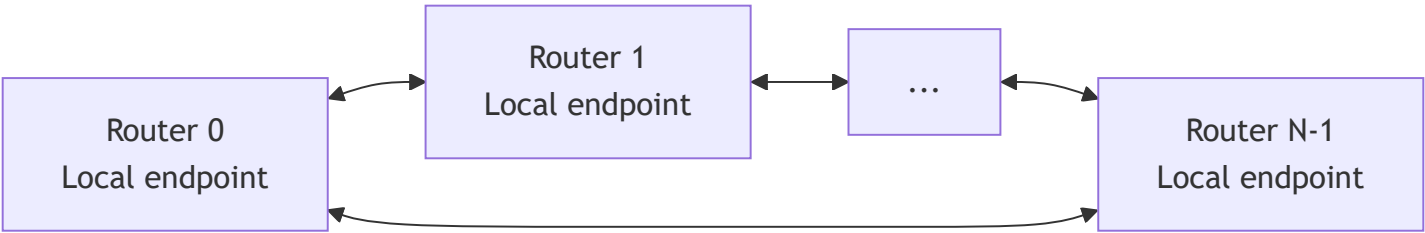
模块	职责
TmRingFabric	创建并连接 NIU、Router、Link、Target Port，提供统一 attach、reset、idle 和统计接口
TmRingTopology	建立节点关系，维护 master/target 到 router 的映射，完成地址译码和路由方向计算
TmRingInf	master 侧 NIU，接收上游事务，分配元数据，管理 master OSD，注入请求并回收响应

模块	职责
TmRingRouter	在 LOCAL/EAST/WEST 输入输出之间转发事务，执行 traffic-class 与端口仲裁
TmRingLink	建模单向、逐跳的链路传播延迟、序列化带宽、FIFO 和最大在途包数量
TmRingTargetPort	接收 Ring 请求，排队并发射到 target，将 target 响应重新注入响应网络
TmBusFlowCtrl	管理全局 OSD、target slot credit、带宽 token、target busy 和热点惩罚
Performance Monitor	汇总端到端延迟、吞吐、公平性、队列/链路占用和各类 stall

4. 拓扑模型

4.1 参数化节点

模型根据配置的 master 和 target 数量建立 Ring 节点。每个 endpoint 通过本地端口连接到一个 router，router 之间建立 EAST 和 WEST 两组有向 link，形成双向闭环。



当前自动放置策略是将 target 尽量均匀分布在 Ring 上，再用 master 填充剩余节点。架构上应保留显式 placement 配置能力，以支持评估不同物理布局 and 平均跳数。

4.2 路由策略

Router 比较目标节点在 EAST/WEST 两个方向上的 hop 数，选择最短路径。距离相同时使用固定方向作为确定性 tie-break。

```
east_hops = (dst + router_count - current) % router_count
west_hops = (current + router_count - dst) % router_count

route = EAST, if east_hops <= west_hops
        WEST, otherwise
```

请求根据 target router 路由，响应根据源 master router 返回。

5. 协议与通道模型

5.1 事务类型

模型支持以下事务类别：

- RD：读请求；
- WR：写地址/写命令请求；
- WR_DAT：写数据；
- RD_RSP：读数据响应；
- WR_RSP：写命令 grant/响应；
- RSP：写数据最终完成响应。

写事务采用命令与数据分离的两阶段模型，使写地址仲裁、写数据传输和最终完成能够分别受到带宽、FIFO 和反压约束。

5.2 Request/Response 子网

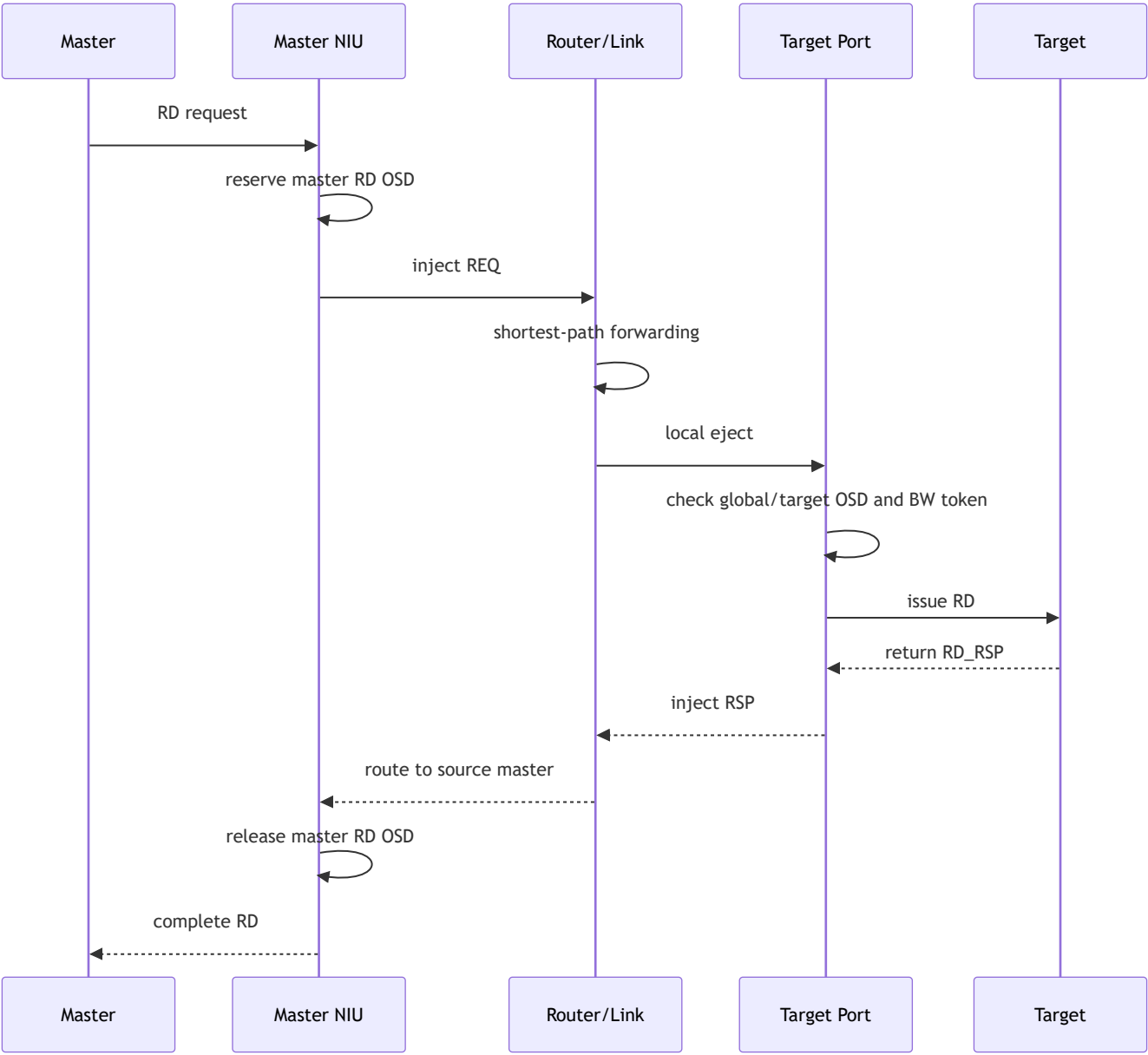
REQ 和 RSP 使用独立逻辑 subnet：

- REQ 承载 RD/WR/WR_DAT；
- RSP 承载 RD_RSP/WR_RSP/RSP；
- 每个 subnet 独立维护链路 FIFO、序列化状态和 inflight 计数；
- 读响应可以配置多个 lane，以表达 target 的并行响应能力。

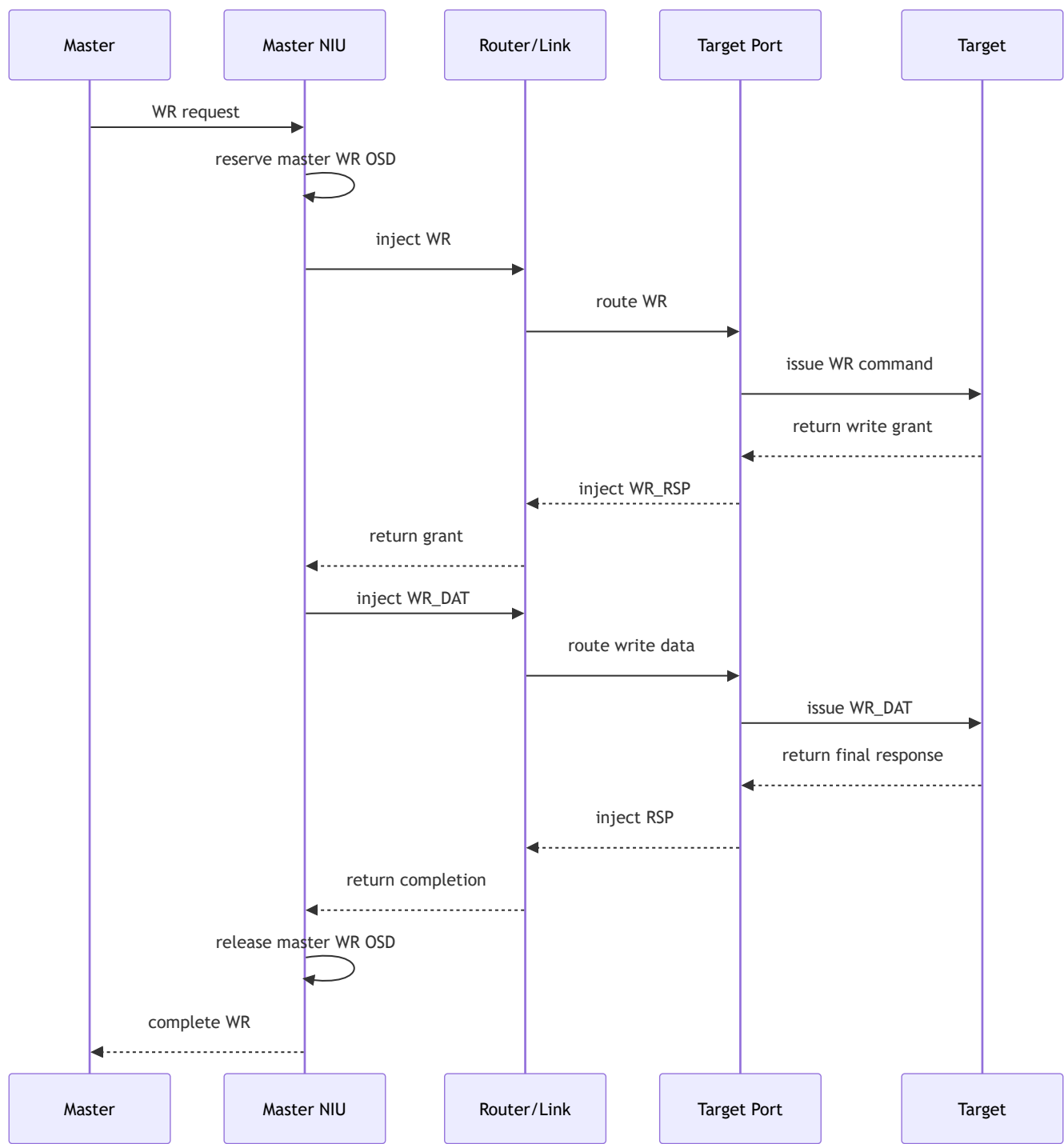
REQ/RSP 分离可以减少响应被请求完全阻塞的风险，但两者是否对应独立物理线宽应由配置和产品定义明确。

6. 读写事务路径

6.1 读事务



6.2 写事务



事务必须携带可唯一关联请求与响应的 ID、source master、目标节点、地址、大小和响应 lane 等元数据。写 grant 可改变局部 transaction ID 时，应保留端到端不变的 global ID。

7. 地址译码与交织

每个 target 配置地址起始位置、地址空间大小和可选交织属性。地址首先通过 target address range 过滤，然后计算 interleave slice。

令：

- A：事务地址；
- B：target 交织地址空间起始地址；
- S：交织粒度 `interleave_size`；
- N：参与交织的 target 数量；
- $\text{stripe} = \text{floor}((A - B) / S)$ 。

LINEAR：

```
target = stripe % N
```

XOR_HASH：

```
target = (stripe ^ (stripe >> hash_shift) ^ hash_seed) % N
```

LINEAR 适合连续地址轮转；XOR_HASH 将高位地址特征折叠到 target 选择位，可缓解多个 master 具有固定地址步长时的同步热点。hash_shift 和 hash_seed 必须可配置，且映射函数应在模型、软件和硬件规格之间保持一致。

8. Router 模型

8.1 端口结构

每个 Router 包含三个方向：

- LOCAL：连接本地 master NIU 或 target port；
- EAST：连接顺时针相邻 Router；
- WEST：连接逆时针相邻 Router。

每个输入按 subnet 和 traffic class 检查 valid 事务，计算输出方向，只有输出可接收时才提交。

8.2 仲裁

traffic class 使用 round-robin，避免读请求、写命令、写数据或不同响应类型发生固定优先级饥饿。建议完整架构采用两级仲裁：

1. 输入级：从当前输入的多个 traffic class 中选择候选事务；
2. 输出级：从竞争同一 output/subnet 的多个输入候选中 round-robin 选择唯一 winner。

输出仲裁指针应仅在事务成功发送后更新，从而保证确定性、公平性和可重放性。后续如需 QoS，可将第二级仲裁扩展为 weighted round-robin 或 age-based arbitration。

8.3 提交规则

Router 必须遵守：

```
route_ready == true
AND downstream_send == success
THEN pop input payload and update arbitration state
ELSE retain input payload
```

这样可以避免下游 FIFO/link 满时的数据丢失。

9. Link 延迟与带宽模型

每个物理方向由独立 TmRingLink 建模，并为 REQ/RSP subnet 分别保存状态。

9.1 固定传播延迟

事务进入 link 后，在 link_latency 个周期后变为可向下游发送。该参数表达布线、寄存器切分或跨模块传输产生的逐跳固定延迟。

9.2 序列化带宽

链路宽度为 link_width_bytes，包占用链路的周期数为：

```
serialization_cycles = max(1, ceil(packet_bytes / link_width_bytes))
next_send_time = current_time + serialization_cycles
```

packet_bytes 根据事务类型确定：命令类事务使用 header 大小，写数据和读响应使用 payload 大小，轻量响应使用 response header 大小。

9.3 缓冲与在途限制

- ring_fifo_depth：link 内部 ready/inflight queue 容量；
- link_max_inflight：已被 link 接收但尚未成功送入下游的最大包数量；
- link_max_inflight 不改变物理带宽，只改变链路能够吸收的突发数量；
- 下游接口满时，ready packet 保留在 link queue 中并持续重试。

建议把链路阻塞拆分为：

- `serialization_busy_stall` ；
- `link_fifo_full_stall` ；
- `link_inflight_full_stall` ；
- `downstream_inf_full_stall` 。

10. Target 端口模型

Target Port 是 Ring 与存储/外设目标模型之间的适配层，负责：

- 将 Ring 请求按 RD、WR、WR_DAT 分别入队；
- 检查 global/target OSD、slot credit 和带宽 token；
- 根据 target issue busy time 控制发射节拍；
- 接收 target 响应并施加响应侧 busy time；
- 将响应注入 Ring RSP subnet。

请求侧忙周期：

```
issue_busy_cycles = frontend_latency
                  + forward_latency
                  + header_latency
                  + ceil(payload_size / target_width)
                  + hotspot_penalty
```

响应侧忙周期：

```
response_busy_cycles = response_latency
                    + header_latency
                    + ceil(response_payload_size / target_width)
                    + hotspot_penalty
```

外部 target 自身的处理延迟由 target 模型负责，Ring Target Port 不应重复计算同一段延迟。

11. 流控与 OSD 架构

11.1 Master OSD

每个 master 独立维护：

- master_rd_osd：最大未完成读事务数；
- master_wr_osd：最大未完成写事务数。

OSD 在 NIU 接受新事务时预留，在完整终态响应返回时释放。命令、grant 和数据阶段不能重复占用同一个写事务的 OSD。

11.2 Global OSD

global_osd 限制整个 fabric 中允许进入受控目标域的未完成事务总数，用于表达共享 tracker、跨 target 公共队列或系统级 transaction table 的容量。

需要冻结其生效位置：

- source admission：事务进入 Ring 前预留，可直接限制网络注入；
- target admission：事务到达 target port 后预留，可限制 target 系统，但请求可能已经占用 Ring。

当前实现采用 target admission。若硬件的 global tracker 位于源端或 fabric ingress，应改为 source admission 或实现端到端 credit 预留。

11.3 Target OSD/slot

每个 target 独立维护：

- target_rd_osd：读事务 slot；
- target_wr_osd：写事务 slot；
- target_acc_osd：读写共享总 slot。

读事务同时消耗 RD 和 ACC slot；写命令同时消耗 WR 和 ACC slot；事务终态响应返回后释放。建议统一规定 OSD 单位为“完整事务数”，避免与 byte credit 或 buffer entry 大小混淆。

11.4 Bandwidth token

每个 target 可以配置 RD、WR 和 aggregate token：

- 事务发送时按 payload byte 消耗 token；
- token 按 token_update_period 周期补充；
- token 最大值控制 burst，补充速率控制长期平均带宽；
- WR command 不消耗数据带宽 token，WR_DAT 消耗写带宽 token。

11.5 热点惩罚

当 target outstanding 超过 hotspot_threshold 后，在 target issue/response busy time 上增加 hotspot_penalty ，用于近似 bank conflict、控制器排队或热点访问额外开销。

12. Interface 与 FIFO 语义

接口参数统一使用 *_inf_delay 表示时延；创建接口时容量为：

$$\text{interface_capacity} = \text{interface_delay} + 1$$

接口 capacity 表达流水寄存能力，不等价于功能 FIFO。以下资源应独立配置：

- master RD command FIFO；
- master WR command FIFO；
- master WR data FIFO；
- master response FIFO；
- target RD/WR/WR_DAT FIFO；
- Ring REQ/RSP link FIFO。

模型必须避免用同一个参数同时表示 delay、queue depth 和 outstanding，以免调整时延时意外改变系统容量。

13. 配置架构

建议配置分为五组：

配置组	主要参数
Topology	master/target 数量、endpoint placement、route tie-break
Address map	address range、default target、interleave type/size/hash shift/hash seed
Link	REQ/RSP latency、width、header size、FIFO depth、max inflight
Endpoint	interface delay、master/target FIFO、response lane 数量
Flow control	master/global/target OSD、slot、token、hotspot 参数
Target timing	frontend/forward/response/header latency、target width

配置优先级建议为：

```
model defaults < named profile < runtime override
```

模型启动时必须校验：

- endpoint 数量、宽度、FIFO、inflight 和必要 OSD 非零；
- interleave size 合法且 target index 不越界；
- hash shift 小于地址位宽；
- 地址区间不溢出、不产生未定义重叠；
- delay 转换为 capacity 时不溢出；
- token update 和 target credit 参数语义一致。

14. 性能模型与指标

14.1 端到端延迟

单笔事务端到端延迟由下列部分共同组成：

```
T_total = T_master_queue  
          + T_request_route/link  
          + T_target_queue  
          + T_target_service  
          + T_response_route/link  
          + T_master_response
```

模型应统计平均、最小、最大延迟；如用于性能签核，建议进一步提供 P50/P90/P99。

14.2 吞吐

吞吐只统计已完成响应对应的有效载荷：

```
payload_B_per_cycle = completed_payload_bytes / valid_measurement_cycles  
bandwidth_GBps = payload_B_per_cycle * clock_GHz
```

未完成、超时或未排空的事务不得计入“已达成性能目标”的结论。

14.3 理论峰值

简单工程上界可以写为：

```
endpoint_peak = min(active_masters, active_targets)
                * min(link_width, target_width)
```

但 Ring 的真实可达上限还受流量方向、平均 hop、链路复用和截面带宽限制。正式评审建议同时给出：

- endpoint peak：端点并行能力上界；
- topology peak：基于路由和 Ring bisection bandwidth 的拓扑上界；
- measured throughput：模型实际完成吞吐。

利用率的分母必须在评审时明确，不能在不同配置间混用。

14.4 公平性

master 公平性使用 Jain 指数：

$$J = (\sum(x_i))^2 / (n * \sum(x_i^2))$$

其中 x_i 为各 master 的完成带宽。J 越接近 1，长期带宽分配越均衡。

15. 可观测性与瓶颈定位

模型至少应输出以下统计：

Master 级

- 发出/完成事务数；
- read/write outstanding peak；
- 平均、最小、最大及分位延迟；
- 完成带宽和公平性；
- master OSD、输入 FIFO 和响应 FIFO stall。

Link 级

- 每条 link、方向和 subnet 的 packet/byte 数；

- busy cycles、utilization、inflight peak；
- serialization、FIFO full、inflight full、downstream full stall。

Target 级

- 每个 target 的 RD/WR/WR_DAT 数量和带宽；
- outstanding peak、slot/token 使用率；
- target queue、OSD、bandwidth token、hotspot stall。

Fabric 级

- global outstanding peak 和 global OSD stall；
- 平均 hop、方向分布和最热链路；
- dominant bottleneck；
- Router/Link/NIU/Target 的 idle 状态；
- 协议错误、丢包、重复响应和未完成事务。

瓶颈判定不应只比较总 stall 次数，还应结合资源 utilization 和观察窗口，因为同一事务可能在多个周期重复计数 stall。

16. 正确性与活性约束

16.1 事务守恒

对每类事务满足：

$$\text{accepted_requests} = \text{completed_responses} + \text{outstanding_transactions}$$

任何时刻不得出现负 outstanding、重复释放 OSD、未知 response ID 或 payload 丢失。

16.2 写顺序

写命令、grant、写数据和最终响应必须通过稳定 ID 关联。若产品协议要求同 ID 或同 master 内严格顺序，需在 NIU 中配置对应 ordering rule。

16.3 排空条件

模型 idle 必须同时满足：

- 所有 master NIU 请求/响应队列为空；
- 所有 Router 输入为空；
- 所有 Link FIFO 为空且 inflight 为 0；
- 所有 Target Port 队列为空；
- 全局和 target outstanding 为 0；
- 外部 target 无未完成事务。

16.4 无死锁要求

在合法配置下，持续反压可以降低吞吐，但下游恢复接收后系统必须继续前进。建议对 REQ/RSP 依赖关系、写 grant/WR_DAT 依赖和 Ring buffer 饱和场景做专门活性检查。

17. 验证方案

验证应围绕总线自身的功能、性能和活性能力：

验证维度	方法	预期
地址译码	覆盖地址边界、default target 和非法地址	target 选择准确、错误可检测
LINEAR 交织	连续跨 stripe 访问	target 按 slice 周期分布
XOR_HASH	改变高位地址、shift 和 seed	分布符合公式且可重放
Link latency	扫描逐跳 delay	端到端延迟随 hop/delay 增加
Link width	扫描序列化宽度	busy cycles 与 $\text{ceil}(\text{bytes}/\text{width})$ 一致
OSD	分别压低 master/global/target OSD	对应 stall 成为主瓶颈
Token	限制补充速率和 burst	长期带宽受配置约束
反压	压低 FIFO/inflight 或暂停 target	无丢包，恢复后能够排空
公平性	多 master 同时竞争同一输出	无长期饥饿，结果可重放
事务一致性	混合 RD/WR/WR_DAT	计数守恒、ID 正确、无重复响应

18. 已知架构问题与评审决策点

编号	议题	当前模型方向	建议评审结论
A1	模型精度	Transaction-level + cycle approximation	明确与参考模型/RTL 的允许误差
A2	Global OSD 生效点	当前偏 target admission	根据硬件 tracker 位置冻结定义
A3	Target OSD 单位	可能继承外部 memory credit	统一为事务数，显式配置
A4	Router 仲裁	traffic-class RR 已具备	增加显式 output-port RR
A5	REQ/RSP 资源关系	逻辑 subnet 独立	明确是否共享物理宽度/FIFO
A6	理论峰值	endpoint 工程上界	增加 topology-aware peak
A7	链路瓶颈统计	部分原因对外聚合	拆分到具体 link/资源/原因
A8	节点放置	自动均匀放置	增加显式 placement 配置
A9	测量窗口	可使用完整事务窗口	增加 warm-up/steady-state/drain
A10	死锁规避	依赖 REQ/RSP 分网和反压推进	给出依赖图并补充活性断言

19. 建议演进阶段

阶段一：功能模型冻结

- 冻结事务类型、地址译码、交织和读写状态机；
- 冻结 delay、FIFO、inflight 和 OSD 参数语义；
- 完成事务守恒、idle 和反压恢复检查；
- 提供独立于 SoC 的通用 attach 和配置入口。

阶段二：性能模型冻结

- 增加显式 output-port 仲裁；
- 拆分 per-link/per-target stall 和 utilization；
- 冻结 target timing、token 和 hotspot 模型；

- 冻结 endpoint peak、topology peak 和测量窗口定义。

阶段三：模型校准

- 使用架构参考数据或 RTL 仿真进行延迟、带宽和拥塞趋势对比；
- 校准 Router pipeline、Link latency、target timing 和 FIFO/OSD；
- 给出不同流量模式下的误差范围和模型适用边界。

20. 评审建议结论

该方案将 Ring Bus 拆分为 Master NIU、Topology、Router、Link、Target Port 和 Flow Control 六类核心组件，能够独立表达事务功能、逐跳时延、序列化带宽、交织、OSD 和反压行为，满足多发起端、多目标端 ESL 互连模型的基本架构要求。

建议专家重点评审并冻结四项内容：

1. Global/Target OSD 的物理含义、单位和占用/释放位置；
2. REQ/RSP 是独立物理资源还是仅逻辑分网；
3. Router 输出仲裁与无死锁保证；
4. topology-aware 理论峰值和性能测量窗口。

上述口径冻结后，该模型可用于总线参数探索、瓶颈定位和架构性能评估。